

About the Protocol

VulnerableVault is a custom ERC20 staking vault where users deposit VaultToken, earn yield weighted by staking duration, stake size, and total vault balance, then withdraw principal plus accrued rewards. The companion VaultToken contract controls the underlying asset supply.

Threat model: Attack surface spans three layers — the token contract (unconstrained mint/burn), the vault's yield oracle (live balanceOf drives payout math), and the withdrawal gate (zero-amount bypass). An attacker with zero capital can fully drain the vault in under 20 transactions; any token holder can permanently lock innocent depositors with a single direct transfer.

Audit scope: `src/Vault.sol`, `src/VaultToken.sol` — commit

a5c86bd27cced400d7ed16f5b64661c913d7b1f2 (2025-06-28). 8 findings: 4 Critical, 1 High, 2 Med

Summary Table

ID	Title	Severity	Status
C-01	Unrestricted Mint/Burn on VaultToken	Critical	Unmitigated
C-02	Zero-Stake Vault Drain via <code>withdraw(0)</code>	Critical	Unmitigated

C-03	Balance Inflation via Direct Token Transfer	Critical	Unmitigated
C-04	Vault Insolvency Permanently Locks All Stakers	Critical	Unmitigated
H-01	Share/Token Unit Mismatch Causes Irrecoverable Deposit Loss	High	Unmitigated
M-01	No Solvency Guard — Vault Self-DoSes as Yield Grows Over Time	Medium	Unmitigated
M-02	Staking Timer Reset Destroys Accumulated Yield on Deposits	Medium	Unmitigated
L-01	Stake Amount vs Share Count	Low	Unmitigated

	Conflation in Withdrawal Guard		
L-02	No Events Emitted for Core Protocol Actions	Low	Unmitigated
L-03	Unchecked ERC20 Transfer Return Values	Low	Unmitigated

Critical Findings

C-01: Unrestricted Mint/Burn on VaultToken

Severity: Critical

Status: Unmitigated

Code: `src/VaultToken.sol#L11-L18`

Description:

`VaultToken` exposes completely unguarded external `mint()` and `burn()` functions with no access control. Any address can mint unlimited tokens to themselves for free, or burn tokens from any address—including the vault itself—without authorisation. This makes the vault trivially exploitable with zero capital and in a single transaction.

Attack Flow:

- Attacker calls `vtoken.mint(attacker, 1_000_000e18)` — free tokens, zero cost.
- Attacker approves vault, deposits, dilutes all legitimate shares, then withdraws extracting all depositor funds.
- Alternatively: `vtoken.burn(address(vault), vaultBalance)` zeros vault token holdings instantly—all staker withdrawals revert permanently.

Recommendations:

- Add `onlyOwner` to `mint()`, or gate it via a `MINTER_ROLE` with `AccessControl`.
- Remove external `burn()` or restrict to the caller's own balance via `OZ ERC20Burnable`.

C-02: Zero-Stake Vault Drain via `withdraw(0)`

Severity: Critical

Status: Unmitigated

Code: `src/Vault.sol#L57-L69, L74-L88`

Description:

`withdraw()` accepts `_amount = 0`. The guard `require(stakedAmount >= _amount)` evaluates `0 >= 0 = true` for any address, including those that have never deposited. `calculateYield()` unconditionally returns `balanceBasedBonus (getVaultBalance() * 5 / 100)` as a flat component regardless of whether the caller has any stake. Any address can therefore call `withdraw(0)` repeatedly and drain 5% of the vault per call with zero capital. 20 calls drain ~64%; 100 calls ~99%.

Attack Flow:

- Any address (no deposit required) calls `vault.withdraw(0)`.
- `require(0 >= 0)` passes; `calculateYield()` returns `0 + balanceBasedBonus`.
- `token.transfer(attacker, 5% of vault balance)` executes.
- Repeat until vault is drained.

Recommendations:

- Add `require(_amount > 0)` as the first line of `withdraw()`.
- Add `require(stakes[msg.sender].amount > 0, "No stake")` before calling `calculateYield()`.

Note: calling `calculateYield()` directly when `totalSupply == 0` also triggers a division-by-zero panic; the `require(stakes[msg.sender].amount > 0)` guard above eliminates this path inside `withdraw()`.

C-03: Balance Inflation via Direct Token Transfer

Severity: Critical

Status: Unmitigated

Code: `src/Vault.sol#L57-L88`

Description:

`getVaultBalance()` returns `token.balanceOf(address(this))`, which includes tokens sent directly to the vault address without going through `deposit()`. `calculateYield()` uses this value as a multiplier for the balance-based bonus, and `withdraw()` applies the result with no ceiling. An attacker holding any shares can inflate the vault's apparent balance via a direct transfer, boosting their yield and draining tokens belonging to other stakers.

Attack Flow:

- Attacker deposits 10 tokens → 10 shares; vault: 110 tokens, 110 totalSupply.
- Attacker waits 6 months.
- Attacker calls `token.transfer(vault, 30)` directly — vault balance becomes 140, totalSupply stays 110.
- `calculateYield()` returns 107 (vs 76 without inflation — a 40% boost).
- Attacker calls `withdraw(10)`: vault transfers $10 + 107 = 117$ tokens. Net profit: +77 on 40 invested.

Recommendations:

- Track deposited balances internally; do not use `token.balanceOf(address(this))` for yield math.
- Add a solvency cap: `require(_amount + yield <= internalBalance)` before the transfer.
- Remove balanceBasedBonus from calculateYield entirely (flagged in code comment at L84).

C-04: Vault Insolvency Permanently Locks All Stakers

Severity: Critical

Status: Unmitigated

Code: `src/Vault.sol#L57-L70`

Description:

`withdraw()` transfers `_amount` + yield with no check that the vault holds sufficient tokens. After the C-03 balance inflation attack drains the vault, the remaining stakers' `calculateYield()` returns claims far exceeding available balance. `token.transfer()` reverts. There is no emergency recovery, rescue function, or pro-rata fallback—every legitimate staker is permanently locked out of their funds.

Attack Flow:

- C-03 executes: vault drained from 140 to 23 tokens.
- Owner holds 100 shares and calls `withdraw(100); calculateYield(owner)` = 159 tokens.
- `withdraw()` attempts `token.transfer(owner, 259)` from a vault holding 23 → **REVERTS**.
- No emergency withdraw or owner rescue function exists. Funds permanently stranded.

Recommendations:

- Add `require(getVaultBalance() >= _amount + yield)` before the transfer.
- Implement an emergency withdrawal that distributes remaining balance pro-rata by shares.
- Separate accounting of deposited principal vs yield-eligible reserves.

High Findings

H-01: Share/Token Unit Mismatch Causes Irrecoverable Deposit Loss

Severity: High

Status: Unmitigated

Code: src/Vault.sol#L40–L53, L57–L70

Description:

`deposit()` mints shares proportional to the current vault balance, but `stakes[msg.sender].amount` stores the raw token deposit amount. `withdraw()` burns `_amount` shares and returns `_amount` + yield tokens, treating share count directly as token quantity. When the vault balance has grown between deposit and withdrawal, the user received fewer shares than tokens deposited. They can only recover the smaller share count in tokens and permanently lose the difference.

Attack Flow:

- Vault: 200 tokens, 100 shares.
- User deposits 10 tokens: $\text{shares} = 10 \times 100 / 200 = 5$. `stakes[user].amount = 10`.
- User calls `withdraw(5)`: burns 5 shares, receives 5 + yield — only 5 of 10 deposited tokens returned.
- User calls `withdraw(10)`: `require(10 >= 10)` passes, `_burn(user, 10)` reverts — only 5 shares exist. 5 tokens permanently locked.

Recommendations:

- On withdrawal compute token return as: $\text{tokens} = \text{shares} \times \text{vaultBalance} / \text{totalSupply}$.
- Standardise `stakes[.].amount` to track shares, not raw token deposit, to eliminate unit confusion.
- Enforce strict 1:1 share minting and prohibit external balance inflation.

Medium Findings

M-01: No Solvency Guard — Vault Self-DoSes as Yield Grows Over Time

Severity: Medium

Status: Unmitigated

Code: `src/Vault.sol#L57-L70, L74-L88`

Description:

`calculateYield()` grows unboundedly with staking duration and has no cap. A legitimate long-term staker will eventually accumulate a yield claim larger than the entire vault balance with no malicious action required. Once any single staker's `_amount` + yield exceeds available tokens, `withdraw()` reverts for that staker and potentially bricks the entire protocol.

Attack Flow:

- Single depositor: 100 tokens, 5-year stake, vault = 100 tokens.
- `calculateYield` returns 7,889 tokens.
- `withdraw(100)` attempts to transfer 7,989 tokens from a vault holding 100 → **REVERTS**.
- No user can withdraw once any single yield claim exceeds vault balance.

Recommendations:

- Cap yield at `available_balance` minus `total_principal_owed`.
- Enforce a maximum **APY** ceiling (e.g. 20% annually) inside `calculateYield`.
- Consider adopting **ERC-4626** which handles yield and share accounting correctly by design.

M-02: Staking Timer Reset Destroys Accumulated Yield on Additional Deposits

Severity: Medium

Status: Unmitigated

Code: `src/Vault.sol#L40-L53`

Description:

`deposit()` unconditionally resets `stakes[msg.sender].startTime` = `block.timestamp` on every call, including top-ups from existing stakers. Any user who adds tokens to an existing position loses all previously accumulated staking duration. There is no yield checkpoint before the reset, so all earned yield is silently erased.

Attack Flow:

- User deposits 100 tokens, waits 180 days, accrues significant yield.
- User deposits 1 additional token to compound position.
- `stakes[user].startTime` = `block.timestamp` – 180 days of accumulated yield erased.
- `calculateYield()` returns only the flat balanceBasedBonus; time-weighted component = 0.

Recommendations:

- Checkpoint yield into `pendingYield[msg.sender]` before resetting `startTime`.
- Or weight-average the start time: $\text{newStart} = (\text{oldTime} \times \text{oldAmount} + \text{now} \times \text{newAmount}) / (\text{oldAmount} + \text{newAmount})$.

Low Findings

L-01: Stake Amount vs Share Count Conflation in Withdrawal Guard

Severity: Low

Status: Unmitigated

Code: `src/Vault.sol#L58-L59`

Description:

The guard `require(stakedAmount >= _amount)` compares `stakes[msg.sender].amount` (deposited tokens) against `_amount` (shares to burn) — two different units that only coincide at a 1:1 share ratio. At any other ratio the check either silently allows burning more shares than exist, or blocks valid share redemptions.

Attack Flow:

- Vault: 200 tokens, 100 shares. User deposits 10 tokens → 5 shares.
`stakes[user].amount` = 10.
- User calls `withdraw(8)`: `require(10 >= 8)` passes, `_burn(user, 8)` reverts — only 5 shares exist.
- User cannot withdraw valid shares; guard misleads about available balance.

Recommendations:

- Compare against `balanceOf[msg.sender]` (share count), not `stakes[msg.sender].amount`.
- Or ensure `_amount` is always denominated in tokens and derive share count internally.

L-02: No Events Emitted for Core Protocol Actions

Severity: Low

Status: Unmitigated

Code: `src/Vault.sol#L40-L70`

Description:

`VulnerableVault` emits no events for deposits, withdrawals, or yield distributions. This prevents off-chain indexing, user transaction history, exploit monitoring, and protocol analytics.

Attack Flow:

- N/A — quality/observability issue.

Recommendations:

- Add event `Deposit(address indexed user, uint256 amount, uint256 shares)` emitted in `deposit()`.
- Add event `Withdraw(address indexed user, uint256 amount, uint256 yield)` emitted in `withdraw()`.

L-03: Unchecked ERC20 Transfer Return Values

Severity: Low

Status: Unmitigated

Code: src/Vault.sol#L52, L69

Description:

`token.transferFrom()` and `token.transfer()` return values are not checked.

`OpenZeppelin` ERC20 reverts on failure so `VaultToken` is safe today, but future composability with non-standard tokens that return false instead of reverting would cause silent accounting corruption.

Attack Flow:

- N/A with standard OZ ERC20. Risk applies if vault token is replaced with non-reverting ERC20.

Recommendations:

- Use `OpenZeppelin` SafeERC20: `token.safeTransfer()` and `token.safeTransferFrom()`.

L-04: Division by Zero in calculateYield When totalSupply Is Zero

Severity: Low

Status: Unmitigated

Code: src/Vault.sol#L85

Description:

`calculateYield()` divides by `totalSupply * scaleFactor`. If `totalSupply` reaches zero—reachable via C-01's unrestricted burn draining all shares—this triggers an unhandled panic revert. As a public view function it also breaks any off-chain integration or monitoring tool that calls it directly.

Attack Flow:

- C-01 attacker calls `vtoken.burn(address(vault), totalShares)` — vault `totalSupply` drops to zero.
- Any call to `calculateYield()` or `withdraw()` reverts with arithmetic panic `0x12`.
- Protocol monitoring and integrations are permanently bricked until redeployment.

Recommendations:

- Add a guard at the top of `calculateYield()`: if (`totalSupply == 0`) return 0;